

Lecture 1

Solving Recursion

→ Substitution method

→ Master method

Lecture note shared. Go through
Coreman book

Lecture 2

Sorting → Searching

→ Quick Sort (Hand written
lecture shared)

→ Heapsort (Coreman)

Lecture 3

Lower bounds of Sorting and Searching

Hand written lecture shared

Lecture 4

Divide and conquer

→ Multiplication of n
digit numbers (lecture note)

→ Counting Inversions (lecture note)

→ Selection (Coreman book)

1 Solving recurrences

Last class we introduced recurrence relations, such as $T(n) = 2T(\lfloor n/2 \rfloor) + n$. Typically these reflect the runtime of recursive algorithms. For example, the recurrence above would correspond to an algorithm that made two recursive calls on subproblems of size $\lfloor n/2 \rfloor$, and then did n units of additional work.

Today we will be learning about how to solve these recurrences to get bounds on the runtime (like $T(n) = O(n \log n)$).

1.1 Substitution method

A lot of things in this class reduce to induction. In the substitution method for solving recurrences we

1. Guess the form of the solution.
2. Use mathematical induction to find the constants and show that the solution works.

1.1.1 Example

Recurrence: $T(1) = 1$ and $T(n) = 2T(\lfloor n/2 \rfloor) + n$ for $n > 1$.

We guess that the solution is $T(n) = O(n \log n)$. So we must prove that $T(n) \leq cn \log n$ for some constant c . (We will get to n_0 later, but for now let's try to prove the statement for all $n \geq 1$.)

As our inductive hypothesis, we assume $T(n) \leq cn \log n$ for all positive numbers less than n . Therefore, $T(\lfloor n/2 \rfloor) \leq c\lfloor n/2 \rfloor \log(\lfloor n/2 \rfloor)$, and

$$\begin{aligned} T(n) &\leq 2(c\lfloor n/2 \rfloor \log(\lfloor n/2 \rfloor)) + n \\ &\leq cn \log(n/2) + n \\ &= cn \log n - cn \log 2 + n \\ &= cn \log n - cn + n \\ &\leq cn \log n \quad (\text{for } c \geq 1) \end{aligned}$$

Now we need to show the base case. This is tricky, because if $T(n) \leq cn \log n$, then $T(1) \leq 0$, which is not a thing. So we revise our induction so that we only prove the statement for $n \geq 2$, and the base cases of the induction proof (which is not the same as the base case of the recurrence!) are $n = 2$ and $n = 3$. (We are allowed to do this because asymptotic notation only requires us to prove our statement for $n \geq n_0$, and we can set $n_0 = 2$.)

We choose $n = 2$ and $n = 3$ for our base cases because when we expand the recurrence formula, we will always go through either $n = 2$ or $n = 3$ before we hit the case where $n = 1$.

So proving the inductive step as above, plus proving the bound works for $n = 2$ and $n = 3$, suffices for our proof that the bound works for all $n > 1$.

Plugging the numbers into the recurrence formula, we get $T(2) = 2T(1) + 2 = 4$ and $T(3) = 2T(1) + 3 = 5$. So now we just need to choose a c that satisfies those constraints on $T(2)$ and $T(3)$. We can choose $c = 2$, because $4 \leq 2 \cdot 2 \log 2$ and $5 \leq 2 \cdot 3 \log 3$.

Therefore, we have shown that $T(n) \leq 2n \log n$ for all $n \geq 2$, so $T(n) = O(n \log n)$.

1.1.2 Warnings

Warning: Using the substitution method, it is easy to prove a weaker bound than the one you're supposed to prove. For instance, if the runtime is $O(n)$, you might still be able to substitute cn^2 into the recurrence and prove that the bound is $O(n^2)$. Which is technically true, but don't let it mislead you into thinking it's the best bound on the runtime. People often get burned by this on exams!

Warning: You must prove the exact form of the induction hypothesis. For example, in the recurrence $T(n) = 2T(\lfloor n/2 \rfloor) + n$, we could falsely "prove" $T(n) = O(n)$ by guessing $T(n) \leq cn$ and then arguing $T(n) \leq 2(c\lfloor n/2 \rfloor) + n \leq cn + n = O(n)$. Here we needed to prove $T(n) \leq cn$, not $T(n) \leq (c+1)n$. Accumulated over many recursive calls, those "plus ones" add up.

1.2 Recursion tree

A recursion tree is a tree where each node represents the cost of a certain recursive subproblem. Then you can sum up the numbers in each node to get the cost of the entire algorithm.

Note: We would usually use a recursion tree to generate possible guesses for the runtime, and then use the substitution method to prove them. However, if you are very careful when drawing out a recursion tree and summing the costs, you can actually use a recursion tree as a direct proof of a solution to a recurrence.

If we are only using recursion trees to generate guesses and not prove anything, we can tolerate a certain amount of "sloppiness" in our analysis. For example, we can ignore floors and ceilings when solving our recurrences, as they usually do not affect the final guess.

1.2.1 Example

Recurrence: $T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2)$

We drop the floors and write a recursion tree for $T(n) = 3T(n/4) + cn^2$.

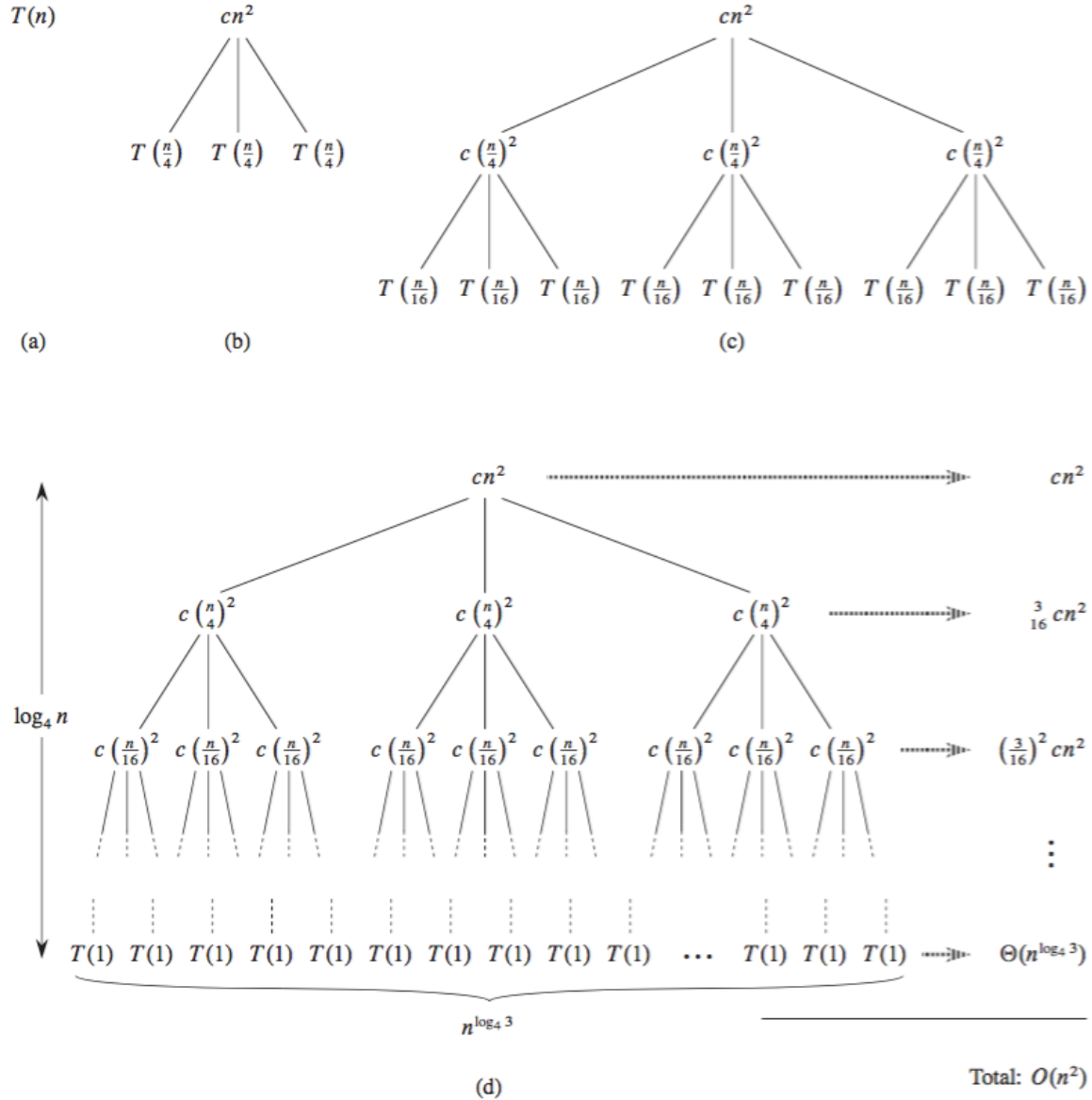


Figure 4.5 Constructing a recursion tree for the recurrence $T(n) = 3T(n/4) + cn^2$. Part (a) shows $T(n)$, which progressively expands in (b)–(d) to form the recursion tree. The fully expanded tree in part (d) has height $\log_4 n$ (it has $\log_4 n + 1$ levels).

The top node has cost cn^2 , because the first call to the function does cn^2 units of work, aside from the work done inside the recursive subcalls. The nodes on the second layer all have cost $c(n/4)^2$, because the functions are now being called on problems of size $n/4$, and the functions are doing $c(n/4)^2$ units of work, aside from the work done inside their recursive subcalls, etc. The bottom layer (base case) is special because each of them contribute $T(1)$ to the cost.

Analysis: First we find the height of the recursion tree. Observe that a node at depth i reflects a subproblem of size $n/4^i$. The subproblem size hits $n = 1$ when $n/4^i = 1$, or

$i = \log_4 n$. So the tree has $\log_4 n + 1$ levels.

Now we determine the cost of each level of the tree. The number of nodes at depth i is 3^i . Each node at depth $i = 0, 1, \dots, \log_4 n - 1$ has a cost of $c(n/4^i)^2$, so the total cost of level i is $3^i c(n/4^i)^2 = (3/16)^i cn^2$. However, the bottom level is special. Each of the bottom nodes contribute cost $T(1)$, and there are $3^{\log_4 n} = n^{\log_4 3}$ of them.

So the total cost of the entire tree is

$$\begin{aligned} T(n) &= cn^2 + \frac{3}{16}cn^2 + \left(\frac{3}{16}\right)^2 cn^2 + \dots + \left(\frac{3}{16}\right)^{\log_4 n - 1} cn^2 + \Theta(n^{\log_4 3}) \\ &= \sum_{i=0}^{\log_4 n - 1} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) \end{aligned}$$

The left term is just the sum of a geometric series. So $T(n)$ evaluates to

$$\frac{(3/16)^{\log_4 n} - 1}{(3/16) - 1} cn^2 + \Theta(n^{\log_4 3})$$

This looks complicated but we can bound it (from above) by the sum of the infinite series

$$\sum_{i=0}^{\infty} \left(\frac{3}{16}\right)^i cn^2 + \Theta(n^{\log_4 3}) = \frac{1}{1 - (3/16)} cn^2 + \Theta(n^{\log_4 3})$$

Since functions in $\Theta(n^{\log_4 3})$ are also in $O(n^2)$, this whole expression is $O(n^2)$. Therefore, we can guess that $T(n) = O(n^2)$.

1.2.2 Back to the substitution method

Now we can check our guess using the substitution method. Recall that the original recurrence was $T(n) = 3T(\lfloor n/4 \rfloor) + \Theta(n^2)$. We want to show that $T(n) \leq dn^2$ for some constant $d > 0$. By the induction hypothesis, we have that $T(\lfloor n/4 \rfloor) \leq d\lfloor n/4 \rfloor^2$. So using the same constant $c > 0$ as before, we have

$$\begin{aligned} T(n) &\leq 3T(\lfloor n/4 \rfloor) + cn^2 \\ &\leq 3d\lfloor n/4 \rfloor^2 + cn^2 \\ &\leq 3d(n/4)^2 + cn^2 \\ &= \frac{3}{16}dn^2 + cn^2 \\ &\leq dn^2 \quad (\text{when } c \leq (13/16)d, \text{ i.e. } d \geq (16/13)c) \end{aligned}$$

Note that we would also have to identify a suitable base case and prove the recurrence is true for the base case, and we don't have time to talk about this in lecture, but you should do that in your homework.

1.3 Master theorem

The master theorem is a formula for solving recurrences of the form $T(n) = aT(n/b) + f(n)$, where $a \geq 1$ and $b > 1$ and $f(n)$ is asymptotically positive. (Asymptotically positive means that the function is positive for all sufficiently large n .)

This recurrence describes an algorithm that divides a problem of size n into a subproblems, each of size n/b , and solves them recursively. (Note that n/b might not be an integer, but in section 4.6 of the book, they prove that replacing $T(n/b)$ with $T(\lfloor n/b \rfloor)$ or $T(\lceil n/b \rceil)$ does not affect the asymptotic behavior of the recurrence. So we will just ignore floors and ceilings here.)

The theorem is as follows:

Theorem 4.1 (Master theorem)

Let $a \geq 1$ and $b > 1$ be constants, let $f(n)$ be a function, and let $T(n)$ be defined on the nonnegative integers by the recurrence

$$T(n) = aT(n/b) + f(n),$$

where we interpret n/b to mean either $\lfloor n/b \rfloor$ or $\lceil n/b \rceil$. Then $T(n)$ has the following asymptotic bounds:

1. If $f(n) = O(n^{\log_b a - \epsilon})$ for some constant $\epsilon > 0$, then $T(n) = \Theta(n^{\log_b a})$.
2. If $f(n) = \Theta(n^{\log_b a})$, then $T(n) = \Theta(n^{\log_b a} \lg n)$.
3. If $f(n) = \Omega(n^{\log_b a + \epsilon})$ for some constant $\epsilon > 0$, and if $af(n/b) \leq cf(n)$ for some constant $c < 1$ and all sufficiently large n , then $T(n) = \Theta(f(n))$. ■

The master theorem compares the function $n^{\log_b a}$ to the function $f(n)$. Intuitively, if $n^{\log_b a}$ is larger (by a polynomial factor), then the solution is $T(n) = \Theta(n^{\log_b a})$. If $f(n)$ is larger (by a polynomial factor), then the solution is $T(n) = \Theta(f(n))$. If they are the same size, then we multiply by a logarithmic factor.

Be warned that these cases are not exhaustive – for example, it is possible for $f(n)$ to be asymptotically larger than $n^{\log_b a}$, but not larger by a polynomial factor (no matter how small the exponent in the polynomial is). For example, this is true when $f(n) = n^{\log_b a} \log n$. In this situation, the master theorem would not apply, and you would have to use another method to solve the recurrence.

1.3.1 Examples:

To use the master theorem, we simply plug the numbers into the formula.

Example 1: $T(n) = 9T(n/3) + n$. Here $a = 9$, $b = 3$, $f(n) = n$, and $n^{\log_b a} = n^{\log_3 9} = \Theta(n^2)$. Since $f(n) = O(n^{\log_3 9 - \epsilon})$ for $\epsilon = 1$, case 1 of the master theorem applies, and the solution is $T(n) = \Theta(n^2)$.

Example 2: $T(n) = T(2n/3) + 1$. Here $a = 1$, $b = 3/2$, $f(n) = 1$, and $n^{\log_b a} = n^0 = 1$. Since $f(n) = \Theta(n^{\log_b a})$, case 2 of the master theorem applies, so the solution is $T(n) = \Theta(\log n)$.

Example 3: $T(n) = 3T(n/4) + n \log n$. Here $n^{\log_b a} = n^{\log_4 3} = O(n^{0.793})$. For $\epsilon = 0.2$, we have $f(n) = \Omega(n^{\log_4 3 + \epsilon})$. So case 3 applies if we can show that $af(n/b) \leq cf(n)$ for some $c < 1$ and all sufficiently large n . This would mean $3\frac{n}{4} \log \frac{n}{4} \leq cn \log n$. Setting $c = 3/4$ would cause this condition to be satisfied.

Example 4: $T(n) = 2T(n/2) + n \log n$. Here the master method does not apply. $n^{\log_b a} = n$, and $f(n) = n \log n$. Case 3 does not apply because even though $n \log n$ is asymptotically larger than n , it is not polynomially larger. That is, the ratio $f(n)/n^{\log_b a} = \log n$ is asymptotically less than n^ϵ for all positive constants ϵ .

1 Heaps

A heap is a type of data structure. One of the interesting things about heaps is that they allow you to find the largest element in the heap in $O(1)$ time. (Recall that in certain other data structures, like arrays, this operation takes $O(n)$ time.) Furthermore, extracting the largest element from the heap (i.e. finding and removing it) takes $O(\log n)$ time. These properties make heaps very useful for implementing a “priority queue,” which we’ll get to later. They also give rise to an $O(n \log n)$ sorting algorithm, “heapsort,” which works by repeatedly extracting the largest element until we have emptied the heap.

1.1 The heap data structure

1.1.1 Intuitive view of the heap

You can view a max heap as a binary tree, where each node has two (or fewer) children, and the key of each node (i.e. the number inside the node) is greater than the keys of its child nodes. (From now on I’m going to say node i is larger than node j when I really mean that the key of i is larger than the key of j .) There are also min heaps, where each node is smaller than its child nodes, but here we will talk about max heaps, with the understanding that the algorithms for min heaps are analogous.

For example, the root of a max heap is the largest element in the heap. However, note that it’s possible for some nodes on level 3 to be smaller than nodes on level 4 (if they’re in different branches of the tree).

The heap is an almost complete binary tree, in that all levels of the tree have to be completely filled except for the level at the bottom. The bottom level gets filled from left to right.

Definition: We define the “height” of a node in a heap to be the number of edges on the longest simple downward path from the node to a leaf. The height of a heap is the height of its root.

Fact: A heap of n nodes has a height of $\lfloor \log n \rfloor$. (Why? Hint: if a heap has height h , what are the minimum and maximum possible number of elements in it? Answer: $2^h \leq n \leq 2^{h+1} - 1$)

Definition: The “max-heap property” (of a heap A) is the property we were talking about, where for every node i other than the root, $A[\text{parent}[i]] \geq A[i]$. The “min-heap property” is defined analogously.

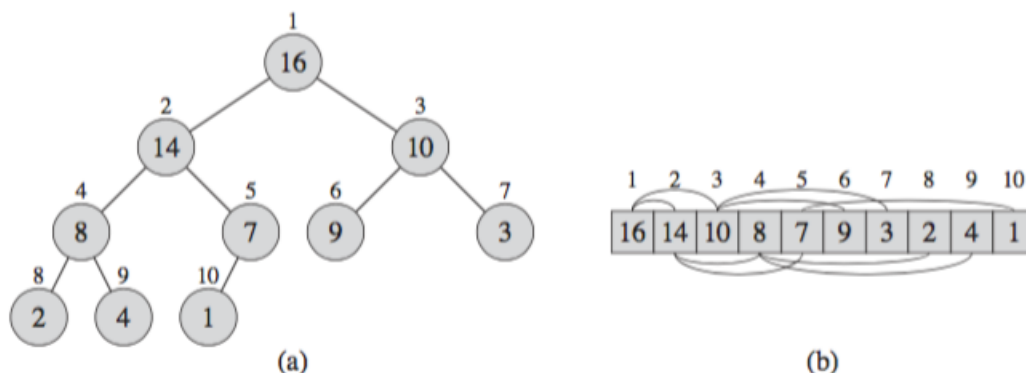


Figure 6.1 A max-heap viewed as (a) a binary tree and (b) an array. The number within the circle at each node in the tree is the value stored at that node. The number above a node is the corresponding index in the array. Above and below the array are lines showing parent-child relationships; parents are always to the left of their children. The tree has height three; the node at index 4 (with value 8) has height one.

1.1.2 Implementation of the heap

You can implement a heap as an array. This array is essentially populated by “reading off” the numbers in the tree, from left to right and from top to bottom.

The root is stored at index 1, and if a node is at index i , then

- Its left child has index $2i$
- Its right child has index $2i + 1$
- Its parent has index $\lfloor i/2 \rfloor$

Furthermore, for the heap array A , we also store two properties: $A.length$, which is the number of elements in the array, and $A.heapsize$, which is the number of array elements that are actually part of the heap. Even though the array A is filled with numbers, only the elements in $A[1..A.heapsize]$ are actually part of the heap.

Fact: The leaves of the heap are the nodes indexed by $\lfloor n/2 \rfloor + 1, \dots, n$.

1.2 Maintaining the heap property

Heaps use this one weird trick to maintain the max-heap property. Unlike most of the algorithms we’ve seen so far, `MaxHeapify` assumes a very specific type of input. Given a heap A and a node i inside that heap, `MaxHeapify` attempts to “heapify” the subtree rooted at i , i.e., it changes the order of nodes in A so that the subtree at i satisfies the max-heap property. However, it assumes that the left subtree of i and the right subtree of i are both max-heaps in and of themselves, so the only possible violation is that i might be smaller than its immediate children.

In order to fix this violation, the algorithm finds which node is largest: i , i 's left child, or i 's right child. If i is largest, the max-heap property is already satisfied. Otherwise, it swaps i with the largest node, which causes the top of the heap to be fine. But this might mess up the subtree that used to be under the largest node, because i might be smaller than some of the nodes in that subtree. So now we call MaxHeapify on that subtree (because the only possible violation of the max-heap property occurs at the top of the subtree).

MaxHeapify runs in time $O(h)$, where h is the height of the node.

Exercise: Show that the worst-case running time of MaxHeapify on a heap of size n is $\Omega(\log n)$.

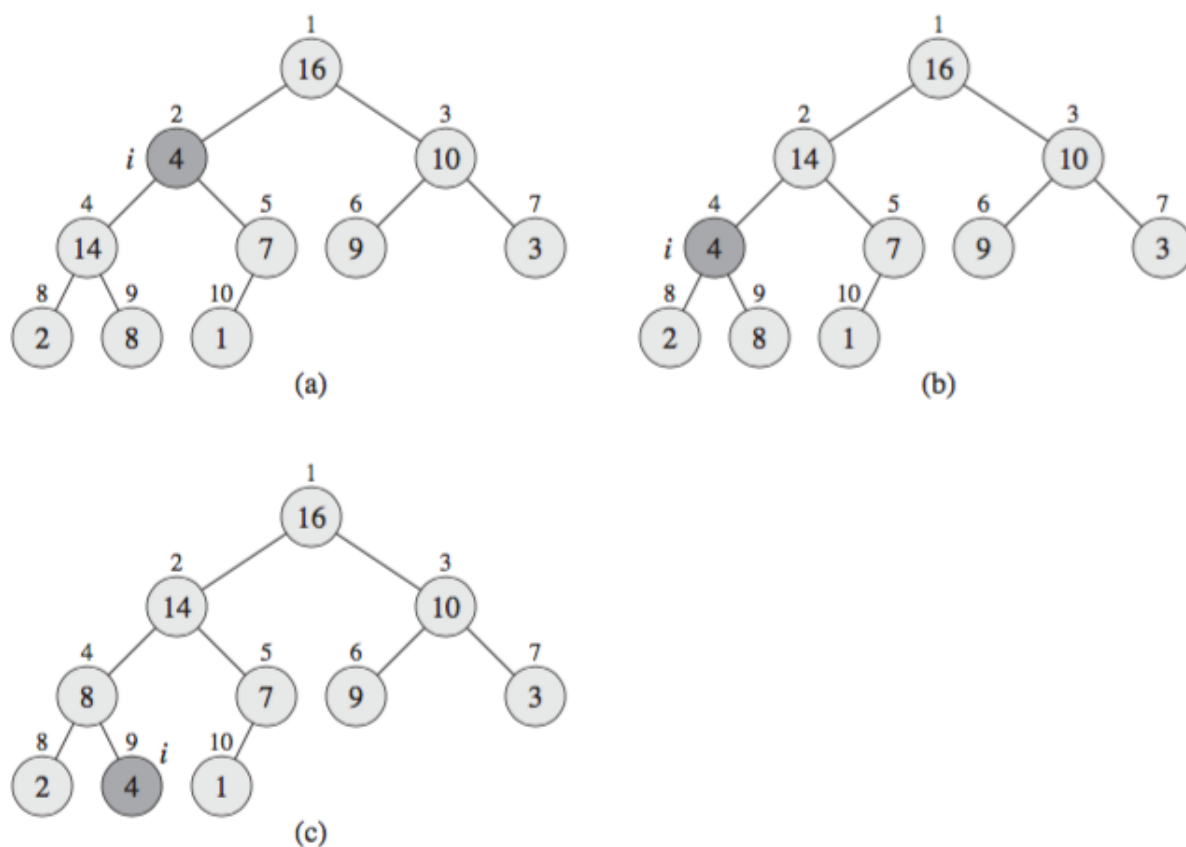


Figure 6.2 The action of $\text{MAX-HEAPIFY}(A, 2)$, where $A.\text{heap-size} = 10$. (a) The initial configuration, with $A[2]$ at node $i = 2$ violating the max-heap property since it is not larger than both children. The max-heap property is restored for node 2 in (b) by exchanging $A[2]$ with $A[4]$, which destroys the max-heap property for node 4. The recursive call $\text{MAX-HEAPIFY}(A, 4)$ now has $i = 4$. After swapping $A[4]$ with $A[9]$, as shown in (c), node 4 is fixed up, and the recursive call $\text{MAX-HEAPIFY}(A, 9)$ yields no further change to the data structure.

Recall that we are implementing heaps as arrays, so the actual code of MaxHeapify takes as input an array A and an index i into that array.

```

MAX-HEAPIFY( $A, i$ )
1   $l = \text{LEFT}(i)$ 
2   $r = \text{RIGHT}(i)$ 
3  if  $l \leq A.\text{heap-size}$  and  $A[l] > A[i]$ 
4       $\text{largest} = l$ 
5  else  $\text{largest} = i$ 
6  if  $r \leq A.\text{heap-size}$  and  $A[r] > A[\text{largest}]$ 
7       $\text{largest} = r$ 
8  if  $\text{largest} \neq i$ 
9      exchange  $A[i]$  with  $A[\text{largest}]$ 
10     MAX-HEAPIFY( $A, \text{largest}$ )

```

1.3 Building a heap

The BuildMaxHeap procedure runs MaxHeapify on all the nodes in the heap, starting at the nodes right above the leaves and moving towards the root. We start at the bottom because in order to run MaxHeapify on a node, we need the subtrees of that node to already be heaps.

Recall that the leaves of the heap are the nodes indexed by $\lfloor n/2 \rfloor + 1, \dots, n$, so when we use our array implementation we just start at the index $\lfloor A.\text{length}/2 \rfloor$ and move towards 1.

```

BUILD-MAX-HEAP( $A$ )
1   $A.\text{heap-size} = A.\text{length}$ 
2  for  $i = \lfloor A.\text{length}/2 \rfloor$  downto 1
3      MAX-HEAPIFY( $A, i$ )

```

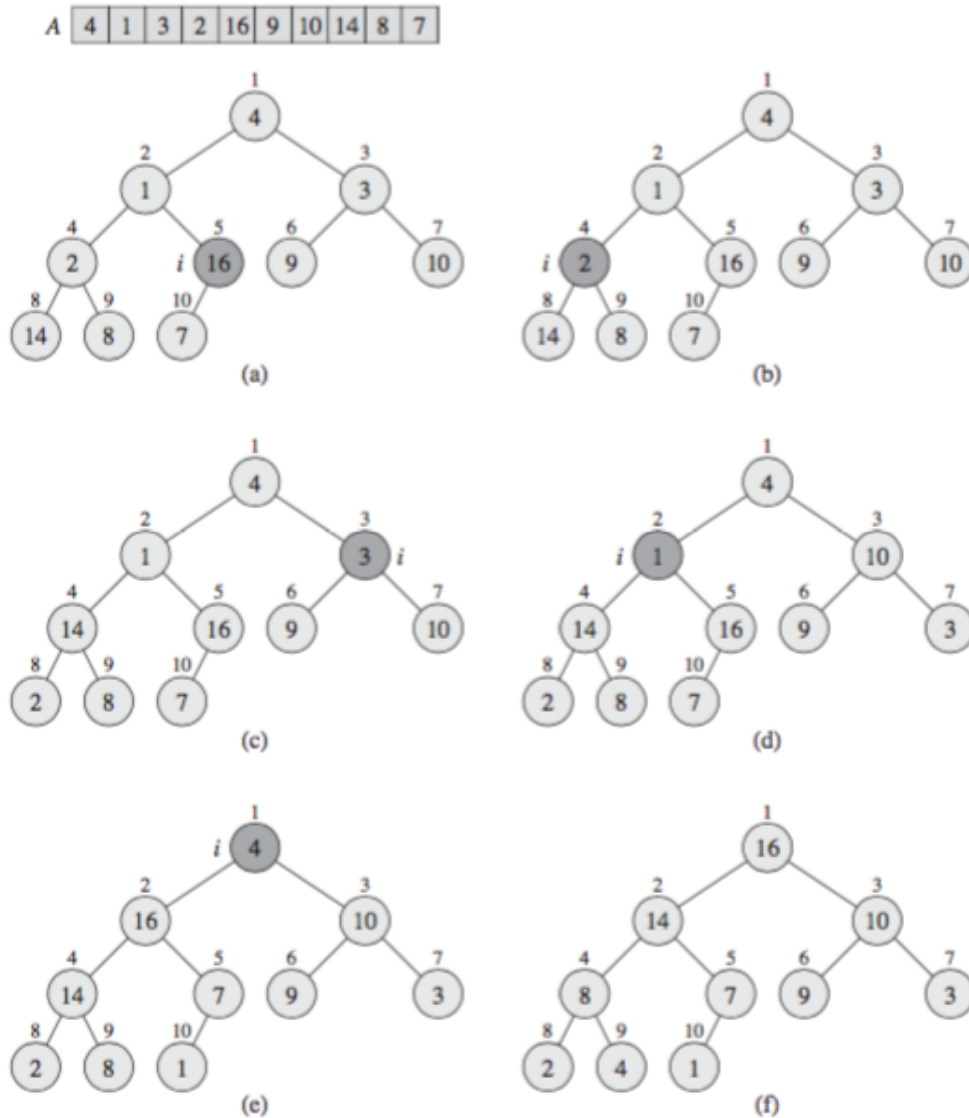


Figure 6.3 The operation of BUILD-MAX-HEAP, showing the data structure before the call to MAX-HEAPIFY in line 3 of BUILD-MAX-HEAP. (a) A 10-element input array A and the binary tree it represents. The figure shows that the loop index i refers to node 5 before the call MAX-HEAPIFY(A, i). (b) The data structure that results. The loop index i for the next iteration refers to node 4. (c)–(e) Subsequent iterations of the for loop in BUILD-MAX-HEAP. Observe that whenever MAX-HEAPIFY is called on a node, the two subtrees of that node are both max-heaps. (f) The max-heap after BUILD-MAX-HEAP finishes.

1.3.1 Correctness

Loop invariant: At the start of each iteration of the for loop, each node $i + 1, i + 2, \dots, n$ is the root of a max-heap.

(Initialization) Prior to the first iteration of the loop, $i = \lfloor n/2 \rfloor$. Each node past that is a leaf and is thus the root of a trivial max-heap.

(Maintenance) To see that each iteration maintains the loop invariant, observe that the children of node i are numbered higher than i . By the inductive hypothesis, they are both roots of max-heaps. This is precisely the condition required for MaxHeapify to make node i the root of a max heap. Moreover, the MaxHeapify call preserves the property that nodes $i + 1, i + 2, \dots, n$ are all roots of max-heaps. Decrementing i in the for loop update reestablishes the loop invariant for the next iteration.

(Termination) At termination, $i = 0$. By the loop invariant, each node $1, \dots, n$ is the root of a max heap. In particular, node 1 is.

1.3.2 Runtime

Basic upper bound: There are $O(n)$ calls to MaxHeapify, each of which takes $O(\log n)$ time. So the running time is $O(n \log n)$.

Better upper bound: Recall that an n -element heap has height $\lfloor \log n \rfloor$. In addition, an n element heap has at most $\lceil n/2^{h+1} \rceil$ nodes of any height h (this is exercise 6.3-3 in the book, and we won't solve it here, but you can prove it by induction).

So the runtime is

$$\sum_{h=0}^{\lfloor \log n \rfloor} \lceil \frac{n}{2^{h+1}} \rceil O(h) = O \left(n \sum_{h=0}^{\lfloor \log n \rfloor} \frac{h}{2^h} \right) \leq O \left(n \sum_{h=0}^{\infty} \frac{h}{2^h} \right) = O(2n) = O(n)$$

Proof of summation formula: Recall that the sum of an infinite geometric series is $\sum_{k=0}^{\infty} x^k = \frac{1}{1-x}$ when $|x| < 1$. Differentiating this formula, we get $\sum_{k=0}^{\infty} kx^{k-1} = \frac{1}{(1-x)^2}$, and multiplying by x , we get

$$\sum_{k=0}^{\infty} kx^k = \frac{x}{(1-x)^2}.$$

Letting $x = 1/2$ makes

$$\sum_{h=0}^{\infty} \frac{h}{2^h} = \frac{1/2}{(1/2)^2} = 2.$$

1.4 Heapsort

Heapsort is a way of sorting arrays. First we use BuildMaxHeap to turn the array into a max-heap. Now we can extract the maximum (i.e. the root) from the heap, swapping it with the last element in the array and then shrinking the size of the heap so we never operate on the max element again. At this point, the heap property is violated because the root may be smaller than other elements, so we call MaxHeapify on the root, which restores the max heap property. Now we can keep repeating this until the whole array is sorted.

HEAPSORT(A)

```
1  BUILD-MAX-HEAP( $A$ )
2  for  $i = A.length$  downto 2
3      exchange  $A[1]$  with  $A[i]$ 
4       $A.heap-size = A.heap-size - 1$ 
5      MAX-HEAPIFY( $A, 1$ )
```

Note that each call to MaxHeapify takes $O(\log n)$ because the height of the heap is $O(\log n)$, so heapsort takes $O(n \log n)$ overall.

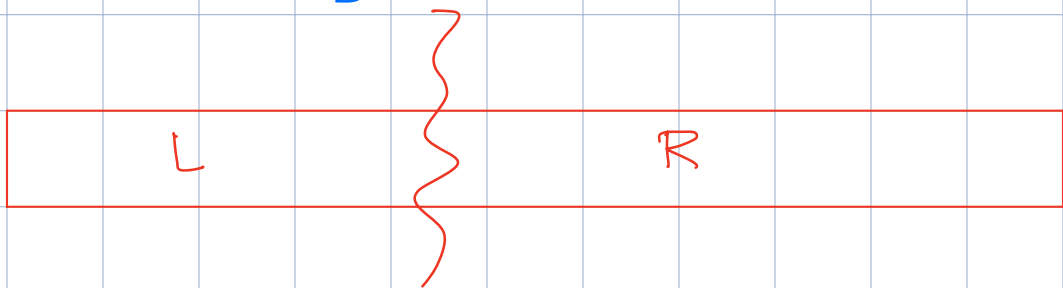
Quick Sort

Quick Sort is one of the popular sorting algorithms.

As usual, we first start with the idea of the algorithm before dive into the actual Pseudocode

Idea

→ Divide the given input array into two part L & R such a way that every element in the part L is less than equal to every element in the part R



→ Now, recursively sort L & R and the entire array will be sorted

How to implement this idea

- choose one element of the array as a pivot. Call this element p
- try to place the elements $\leq p$ towards the left part of the array & the elements $> p$ are placed in the right part
- the above arrangement ensures the partition we talked about in the previous page
- This idea can be easily implemented with the use of $O(n)$ additional space in the following way

* Assume that the first element of the array is chosen as pivot

* use an additional array B

* $j = 1, k = n$

$$3 < 5$$

5	7	1	3	11	6	9
---	---	---	---	----	---	---

↑ p i

1	3					7
---	---	--	--	--	--	---

j k

$$11 > 5$$

5	7	1	3	11	6	9
---	---	---	---	----	---	---

↑ p i

1	3				11	7
---	---	--	--	--	----	---

j k

$$6 > 5$$

5	7	1	3	11	6	9
---	---	---	---	----	---	---

↑ p i

1	3			6	11	7
---	---	--	--	---	----	---

j k

$$9 > 5$$

5	7	1	3	11	6	9
---	---	---	---	----	---	---

↑ p i

1	3		9	6	11	7
---	---	--	---	---	----	---

k, j

Finally, $A(i)$ is placed in the array B

5	7	1	3	11	6	9
---	---	---	---	----	---	---

↑ p i

1	3	5	9	6	11	7
---	---	---	---	---	----	---

j

Final Partition

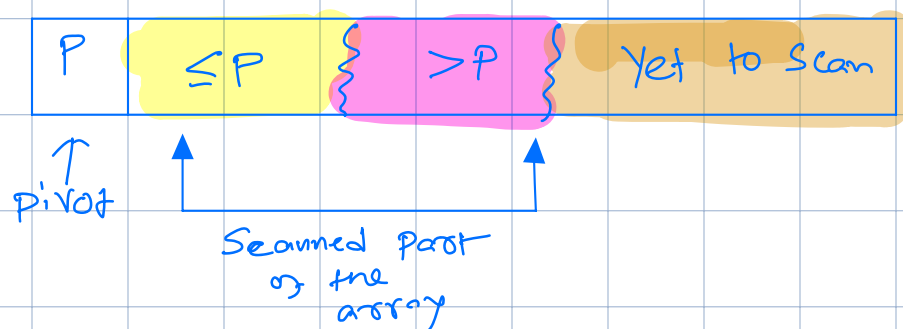
1	3	5	9	6	11	7
---	---	---	---	---	----	---

L R

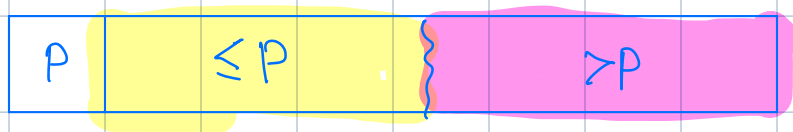
The ultimate plan is to achieve such a partition with only constant amount of additional space.

To achieve this, the plan is to do a single scan of the array (as we did in the previous case with additional memory) and whenever a pair of elements (a_i, a_j) is found where $a_i > P$ and $a_j < P$, then swap the elements.

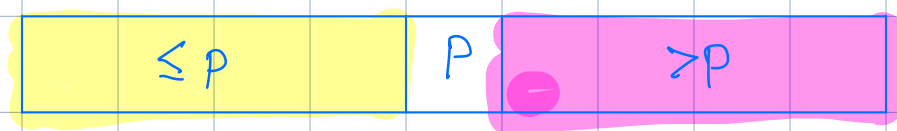
In general, as we scan through the elements, we will take care to ensure that it has the following form:



Finally, when all elements are scanned, then the array looks like following



At the last, Swap P with the last element of the yellow part to achieve the partition that looks like



Final step and the Pseudo Code

→ We maintain two variables i, j in such a way that when j elements are scanned, the subarray 2 to $i-1$ have elements $\leq P$
the subarray i to j have elements $> P$

→ Initialize $j = i = 1$ so before we start scanning,

the subarray 2 to $i-1$ (empty) $\leq P$
the subarray $i+1$ to j (empty)

→ For $j = 2$ to n , whenever an element $A(j)$ is found which is less than P , we swap this element with $A(i)$ (This is because $A(i)$ is the first element that is more than P . We want an element $\leq P$ in the left of an element $> P$)

→ The above step increased the size of the subarray containing the elements $\leq P$. So increment the value of i by 1.

Continue the above steps until the scanning of the entire array is done

Final Pseudocode

Partition(A, l, r)

1. $p \leftarrow A[l]$
2. $i = j = l - 1$
3. for $j = l$ to r
 - if $A[j] \leq p$
 - $i = i + 1$
 - swap($A[i], A[j]$)
4. swap($A[l], A[i+1]$)
5. return i

Quicksort(A, l, r)

1. If $l \geq r$, then return
2. $t \leftarrow \text{Partition}(A, l, r)$
3. Quicksort($A, l, t-1$)
3. Quicksort($A, t+1, r$)

Running time

The running time of the Quick Sort will be calculated from the following recursion relation.

$$T(n) = T(i^0) + T(n-i^0) + n-1$$

where i^0 is the final position of the pivot at the end of the function partition

Since in worst case, the value of i^0 can be 1,

$$\therefore T(n) = T(1) + T(n-1) + n-1$$

$$\Rightarrow T(n) \in O(n^2)$$

Randomized Quick Sort

If a fixed element is chosen as the pivot in the subroutine **partition**, one can always come up with an input for which the array will split into sub arrays of size 1 to $n-1$.

In order to avoid this scenario, instead of choosing a fixed pivot, choose a random element as the pivot in partition. In particular, we made the following changes in the pseudocode of the partition

Partition(A, l, r)

0. $t = \text{random}(l, r)$. swap($A(t), A(r)$)

1. $p \leftarrow A(l)$

2. $i = j = l - 1$

3. for $j = l$ to r

if $A(j) \leq p$

$i = i + 1$

swap($A(i), A(j)$)

4. swap($A(l), A(i+1)$)

5. return i

The rest of the algorithm stays same.

Since a random choice is made inside the algorithm, we need to analyze the average case performance of the algorithm.

Let $T(n)$ be the time for sorting n element array using Quicksort

then each of the following event occurs with probability $\frac{1}{n}$.

$$T(n) = T(1) + T(n-1) + n-1$$

$$T(n) = T(2) + T(n-2) + n-1$$

$$T(n) = T(3) + T(n-3) + n-1$$

⋮
;

$$T(n) = T(n-1) + T(1) + n-1$$

Hence, Expected time complexity (average case) can be expressed as

$$T(n) = \frac{1}{n} \sum_{i=1}^{n-1} T(i) + T(n-i) + n-1$$

$$\text{or, } T(n) = \frac{2}{n} \sum_{i=1}^{n-1} T(i) + n-1 \quad \dots (1)$$

$$\text{Now, } T(n-1) = \frac{2}{n-1} \sum_{i=1}^{n-2} T(i) + n-2$$

$$\therefore \frac{2}{n} \sum_{i=1}^{n-2} T(i) = \frac{n-1}{n} (T(n-1) - n+2)$$

Substituting this value in equation (1)

$$T(n) = \frac{n-1}{n} (T(n-1) - n + 2) + \frac{2}{n} T(n-1) + n - 1$$

$$= \frac{n+1}{n} T(n-1) + 2 \left(\frac{n-1}{n} \right)$$

$$\leq \frac{n+1}{n} T(n-1) + 2$$

$$\leq \frac{n+1}{n} \left\{ \frac{n}{n-1} T(n-2) + 2 \right\} + 2$$

$$= \frac{n+1}{n-1} T(n-2) + 2(n+1) \frac{1}{n} + 2$$

$$\leq \frac{n+1}{n-1} \left\{ \frac{n-1}{n-2} T(n-3) + 2 \right\} + 2(n+1) \frac{1}{n} + 2$$

$$= \frac{n+1}{n-2} T(n-3) + 2(n+1) \left(\frac{1}{n} + \frac{1}{n-1} \right) + 2$$

Continuing in this way we get

$$T(n) \leq (n+1) T(0) + 2(n+1) \left(\frac{1}{n} + \dots + 1 \right) + 2$$

$$= 0 + 2(n+1) H_n$$

$$\in O(n \log n)$$

Divide and Conquer

Divide and Conquer

Divide and Conquer is general algorithmic design framework.

Related to *induction*:

- ▶ Suppose you have a “box” that can solve problems of size $\leq k < n$
- ▶ You use this box on some subset of the input items to get partial answers
- ▶ You combine these partial answers to get the full answer.

But: you construct the “box” by recursively applying the same idea until the problem is small enough to be solved by brute force.

Merge Sort

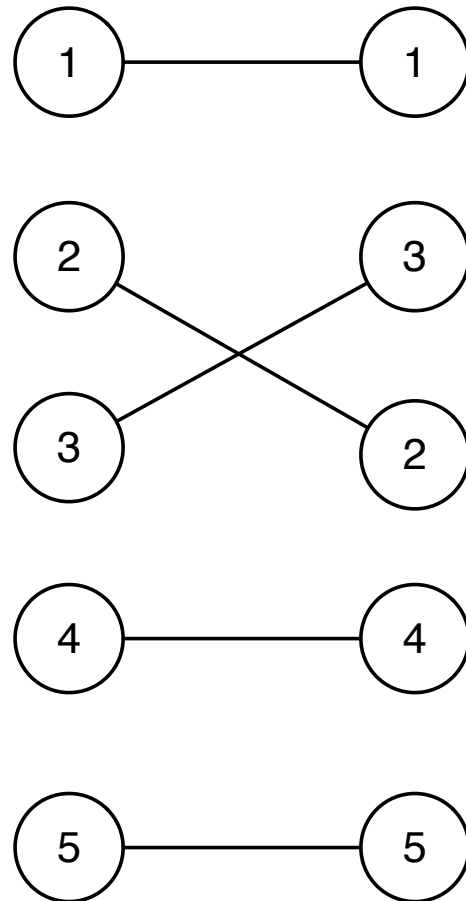
```
MergeSort(L):  
    if |L| = 2:  
        return [min(L), max(L)]  
    else:  
        L1 = MergeSort(L[0, |L|/2])  
        L2 = MergeSort(L[|L|/2+1, |L|-1])  
        return Combine(L1, L2)
```

- ▶ In practice, you sort in-place rather than making new lists.
- ▶ Combine(L1,L2) walks down the sorted lists putting the smaller number onto a new list. Takes $O(n)$ time
- ▶ Total time: $T(n) \leq 2T(n/2) + cn$.

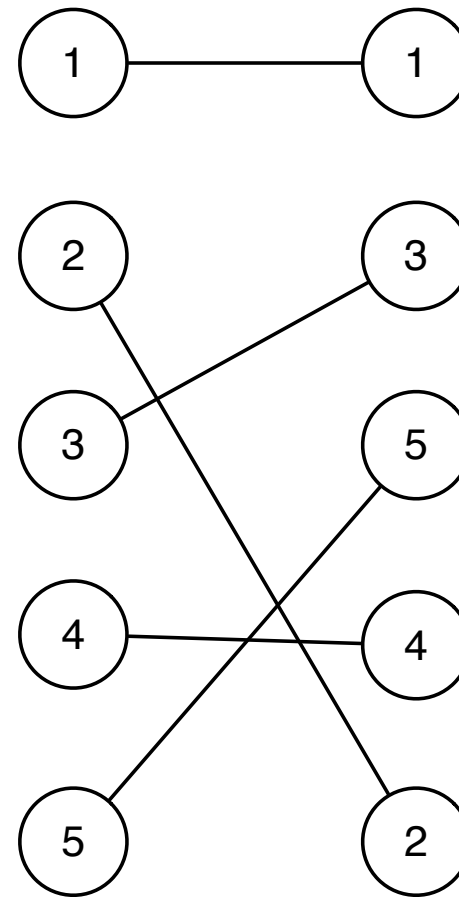
Counting Inversions

Comparing Rankings

Suppose two customers **rank** a list of movies.



similar



more different

A measure of distance

What's a good measure of how dissimilar two rankings are?

A measure of distance

What's a good measure of how dissimilar two rankings are?

We can count the number of inversions:

- ▶ Assume one of the rankings is $1, 2, 3, \dots, n$.
- ▶ Denote the other ranking by $a_1, a_2, \dots, a_n$.
- ▶ An inversion is a pair (i, j) such that $i < j$ but $a_j < a_i$.

Two identical rankings have no inversions.

How many inversions do opposite rankings have?

A measure of distance

What's a good measure of how dissimilar two rankings are?

We can count the number of inversions:

- ▶ Assume one of the rankings is $1, 2, 3, \dots, n$.
- ▶ Denote the other ranking by $a_1, a_2, \dots, a_n$.
- ▶ An inversion is a pair (i, j) such that $i < j$ but $a_j < a_i$.

Two identical rankings have no inversions.

How many inversions do opposite rankings have? $\binom{n}{2}$

How can we count inversions quickly?

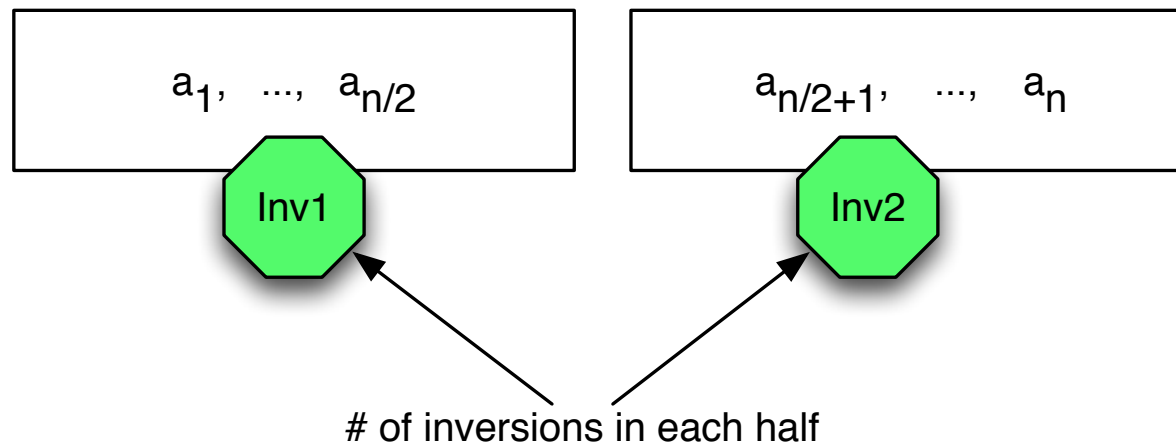
How can we count inversions quickly?

- ▶ **Brute Force:** check every pair: $O(n^2)$.
- ▶ Some sequences might have $O(n^2)$ inversions, so you might think that it might take as much as $O(n^2)$ time to count them.
- ▶ In fact, with divide and conquer, you can count them in $O(n \log n)$ time.

Basic Divide and Conquer

Count the number of inversions in the sequence $a_1, \dots, a_n$.

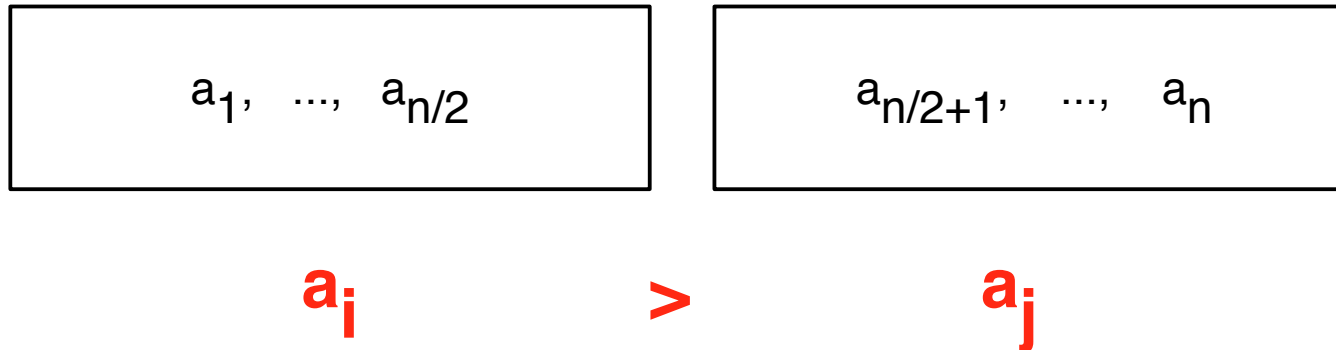
Suppose I told you the number of inversions in the first half of the list and in the second half of the list:



What kinds of inversions are not accounted for in $\text{Inv1} + \text{Inv2}$?

Half-Crossing Inversions

The inversions we have to count during the merge step:

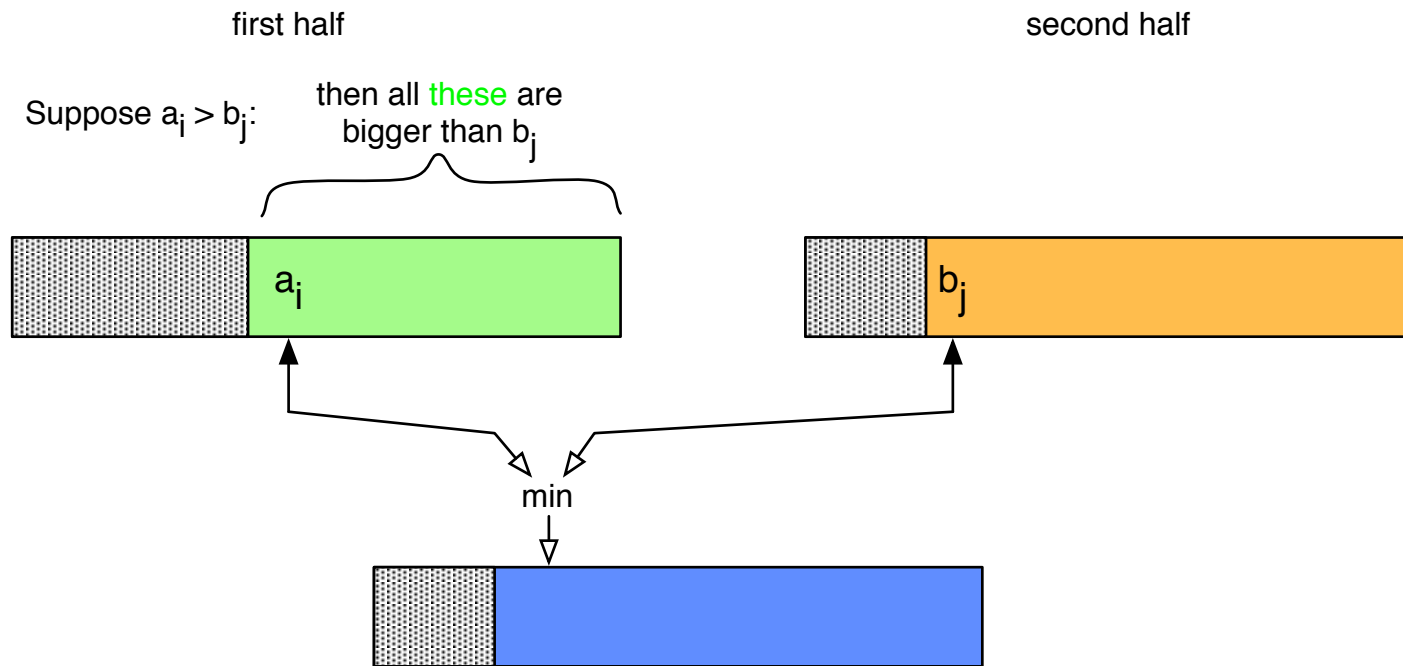


The crux is that we have to count these kinds of inversion in $O(n)$ time.

What if each of the half lists were sorted?

Suppose each of the half lists were sorted.

If we find a pair $a_i > b_j$, then we can infer many other inversions:



Each of the green items is an inversion with b_j .

Merge-and-Count

```
MergeAndCount(SortedList A, SortedList B):
```

```
    a = b = CrossInvCount = 0
```

```
    OutList = empty list
```

```
    While a < |A| and b < |B|:      // not at end of a list
```

```
        next = min(A[a], B[b])
```

```
        OutList.append(next)
```

```
        If B[b] == next:
```

```
            b = b + 1
```

```
            CrossInvCount += |A| - a //inc by # left in A
```

```
        Else
```

```
            a = a + 1
```

```
    EndWhile
```

```
    Append the non-empty list to OutList
```

```
    Return CrossInvCount and OutList
```


Sorted!

Note that MergeAndCount will produce a sorted list as well as the number of cross inversions.

SortAndCount

SortAndCount(List L):

 If $|L| == 1$: Return 0

 A, B = first & second halves of L

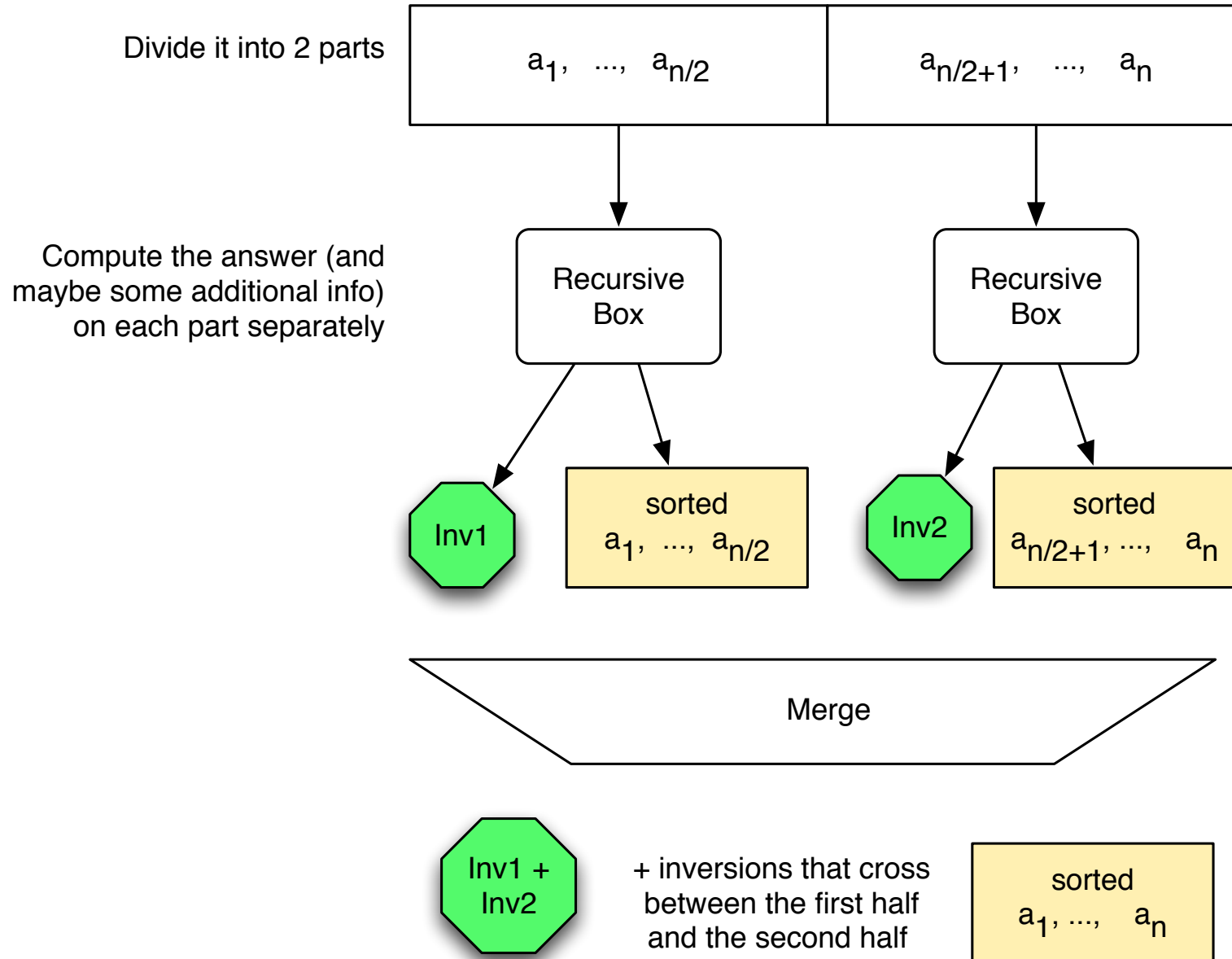
 invA, SortedA = SortAndCount(A)

 invB, SortedB = SortAndCount(B)

 crossInv, SortedL = MergeAndSort(SortedA, SortedB)

 Return $\text{invA} + \text{invB} + \text{crossInv}$ and SortedL

Algorithm Schematic



Running time?

What's the running time of SortAndCount?

Running time?

What's the running time of SortAndCount?

Break the problem into two halves.

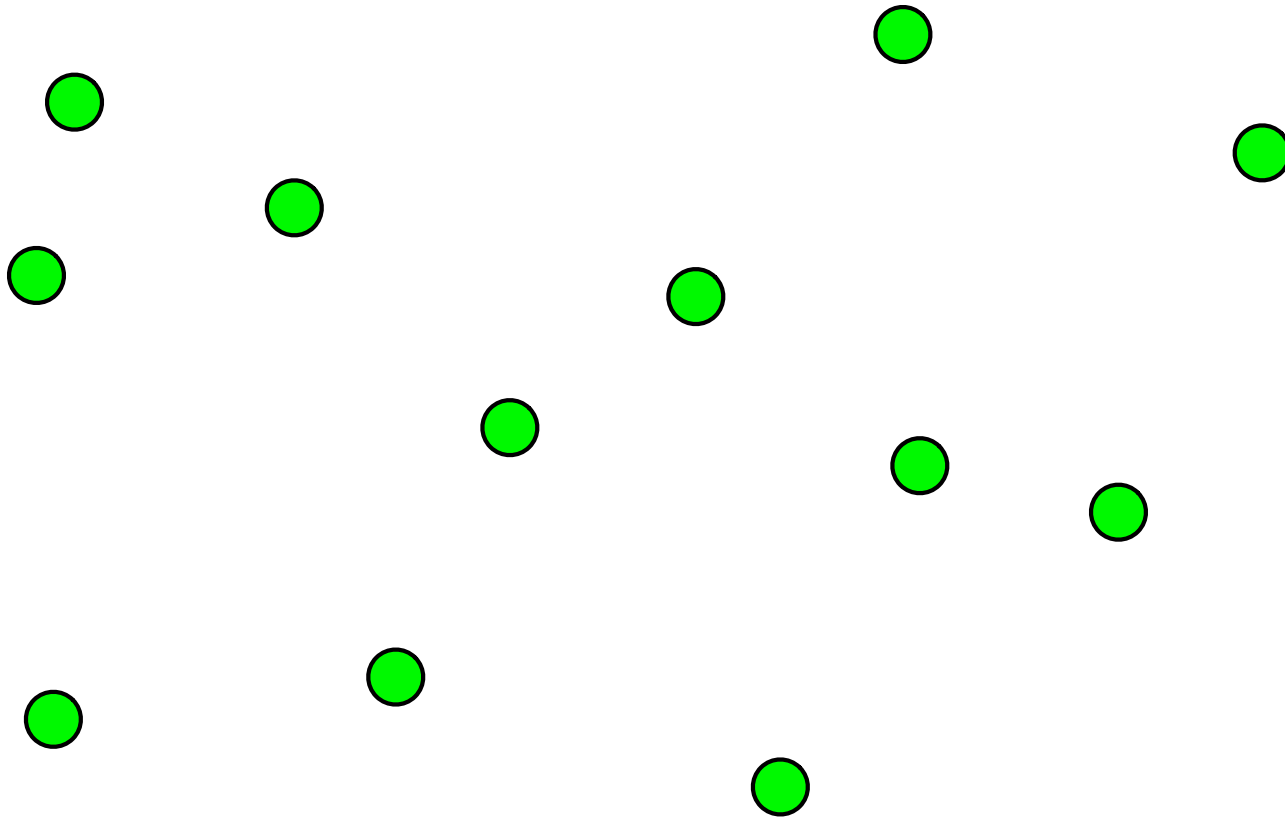
Merge takes $O(n)$ time.

$$T(n) \leq 2T(n/2) + cn$$

$\Rightarrow$ Total running time is $O(n \log n)$.

Finding closest pair of points

Problem Given a set of points $\{p_1, \dots, p_n\}$ find the pair of points $\{p_i, p_j\}$ that are closest together.



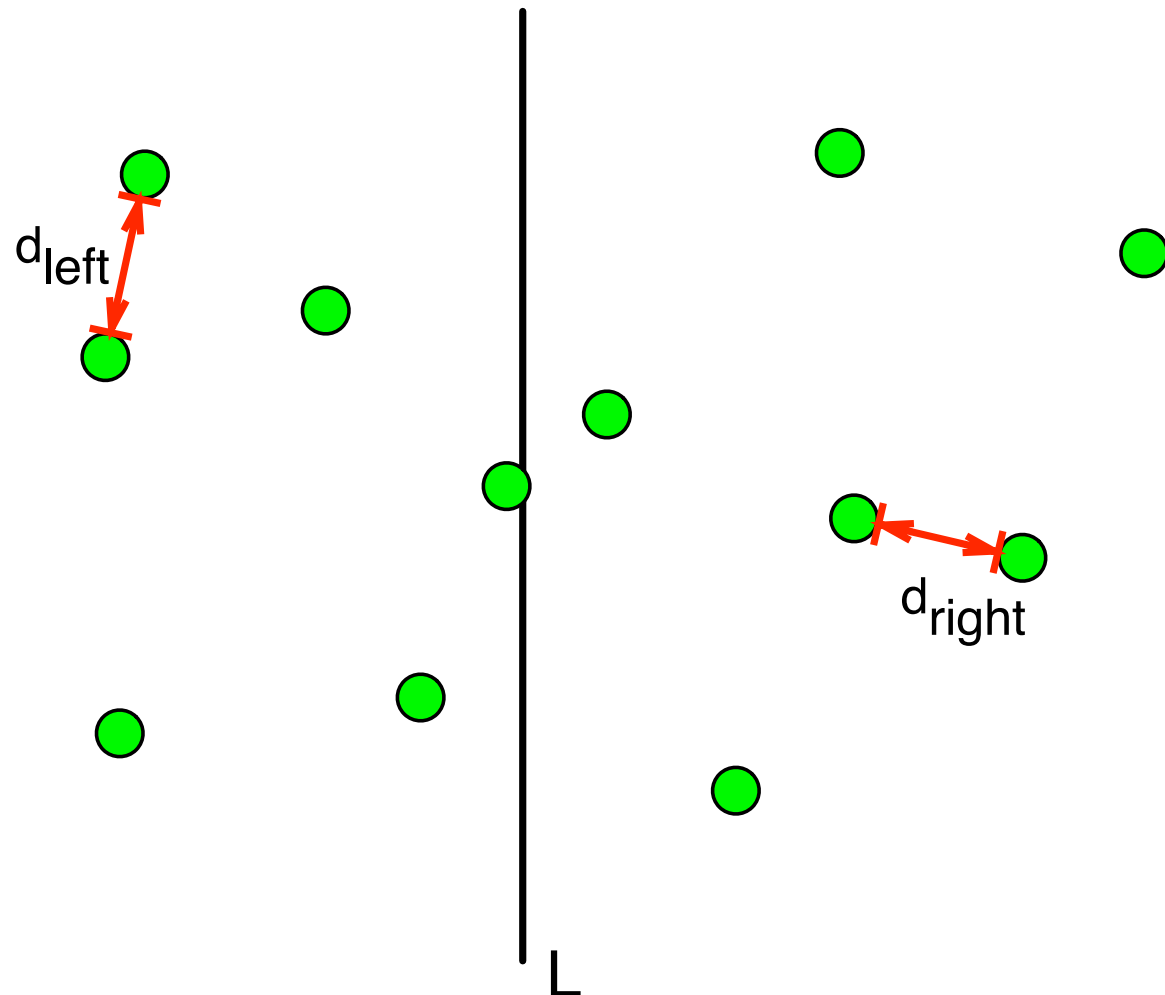
Goal

- ▶ Brute force gives an $O(n^2)$ algorithm: just check every pair of points.
- ▶ Can we do it faster? Seems like no: don't we have to check every pair?
- ▶ In fact, we can find the closest pair in $O(n \log n)$ time.
- ▶ What's a reasonable first step?

Divide

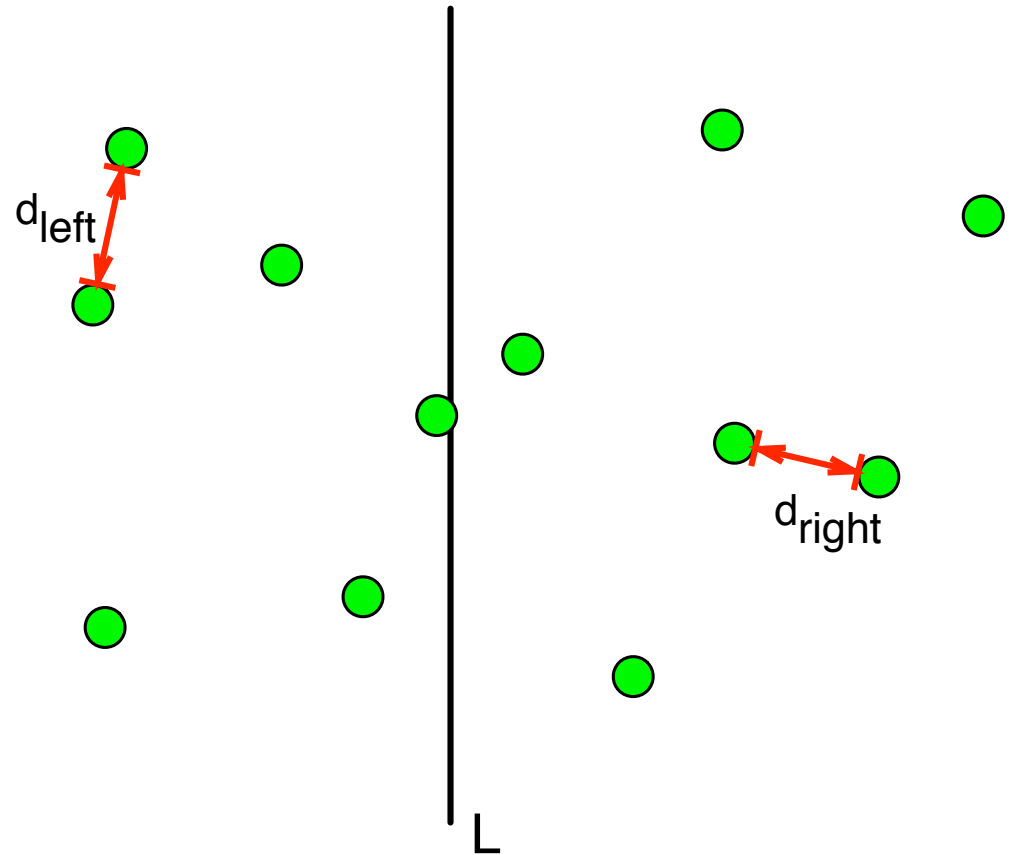
Split the points with line L so that half the points are on each side.

Recursively find the pair of points closest in each half.



Merge: the hard case

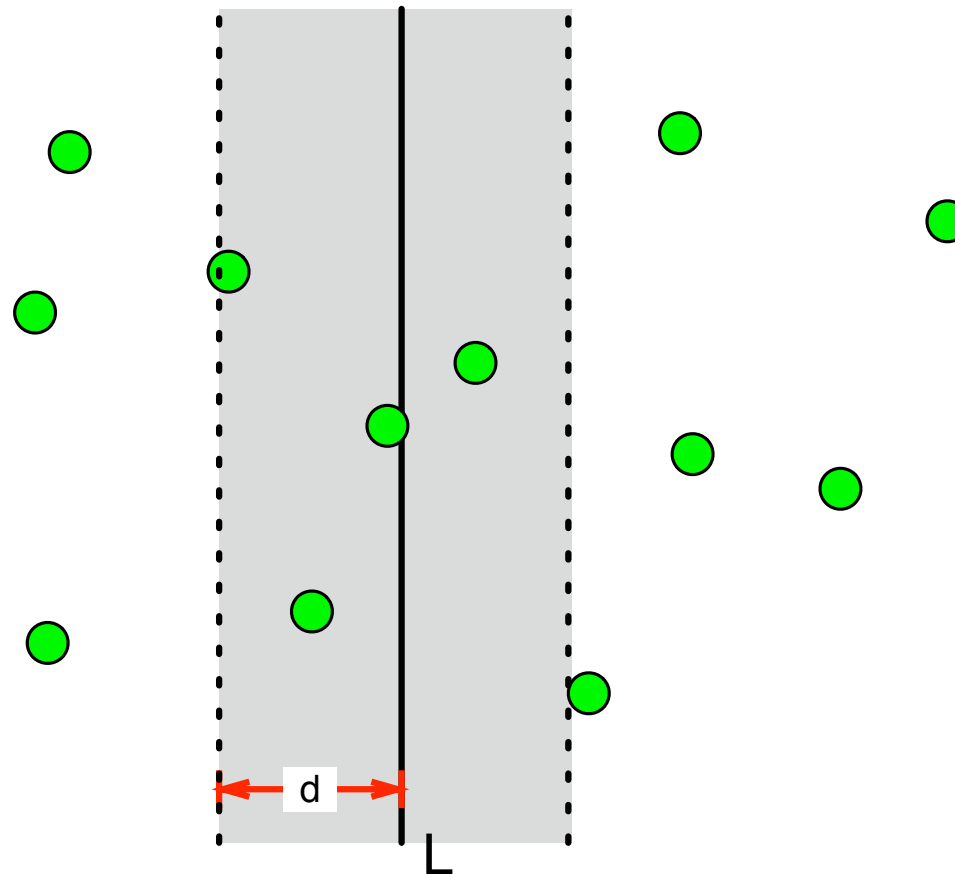
Let $d = \min\{d_{\text{left}}, d_{\text{right}}\}$.



- d would be the answer, except maybe L split a close pair!

Region Near L

If there is a pair $\{p_i, p_j\}$ with $\text{dist}(p_i, p_j) < d$ that is split by the line, then both p_i and p_j must be within distance d of L .



Let S_y be an array of the points in that region, sorted by decreasing y -coordinate value.

Slab Might Contain All Points

- ▶ Let S_y be an array of the points in that region, sorted by decreasing y -coordinate value.
- ▶ S_y might contain all the points, so we can't just check every pair inside it.

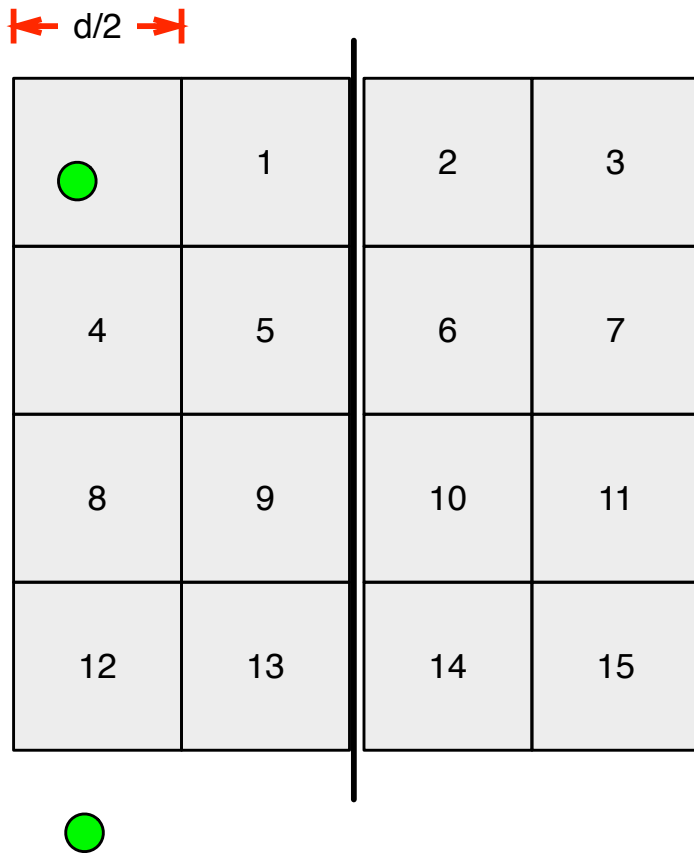
Theorem. Suppose $S_y = p_1, \dots, p_m$. If $\text{dist}(p_i, p_j) < d$ then $j - i \leq 15$.

In other words, if two points in S_y are close enough in the plane, they are close in the array S_y .

Proof, 1

Divide the region up into squares with sides of length $d/2$:

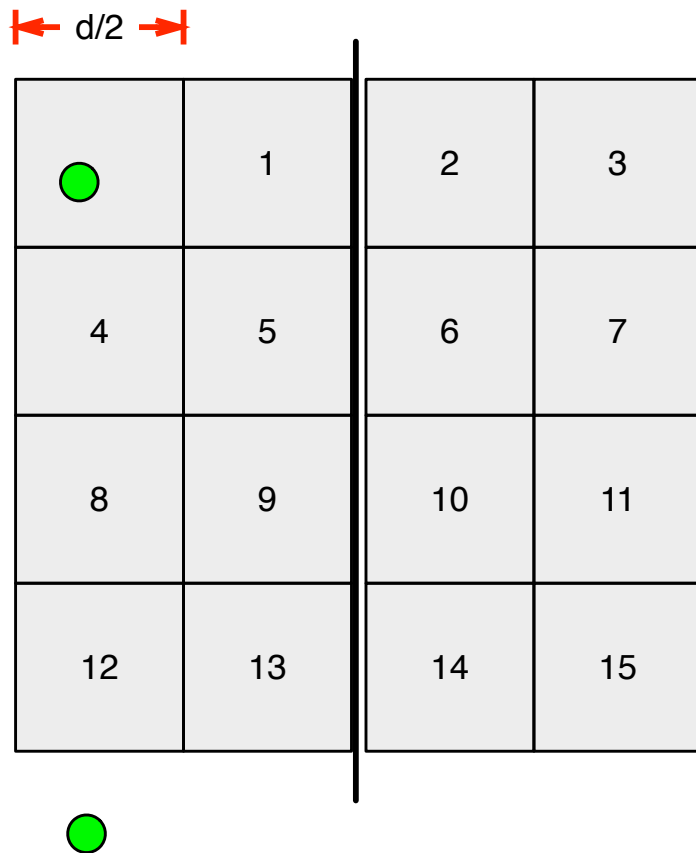
How many points in each box?



Proof, 1

Divide the region up into squares with sides of length $d/2$:

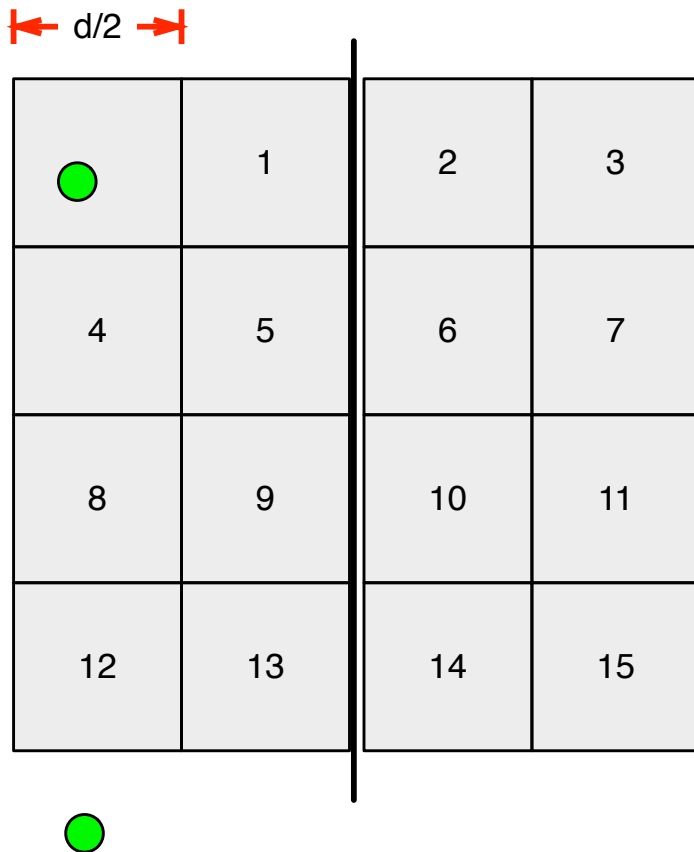
How many points in each box?



At most 1 because each box is completely contained in one half and no two points in a half are closer than d .

Proof, 2

Suppose 2 points are separated by > 15 indices.



- ▶ Then, at least 3 full rows separate them (the packing shown is the smallest possible).
- ▶ But the height of 3 rows is $> 3d/2$, which is $> d$.
- ▶ So the two points are farther than d apart.

SELECTION in LINEAR TIME

Selection

From 9.3 in CLRS

Problem: Given a list A of n of **unordered** distinct keys, and a $i \in \mathbb{Z}, 1 \leq i \leq n$, select the i -smallest element $x \in A$ that is larger than exactly $i - 1$ other elements in A .

Notice if:

1. $i = 1 \Rightarrow$ MINIMUM element
2. $i = n \Rightarrow$ MAXIMUM element
3. $i = \lfloor \frac{n+1}{2} \rfloor \Rightarrow$ the **MEDIAN**
4. $i = \lfloor 0.25 \cdot n \rfloor \Rightarrow$ *order statistics*

Non smart approach:

Sort A in $(O(n \lg n))$ steps and search for $A[k]$.

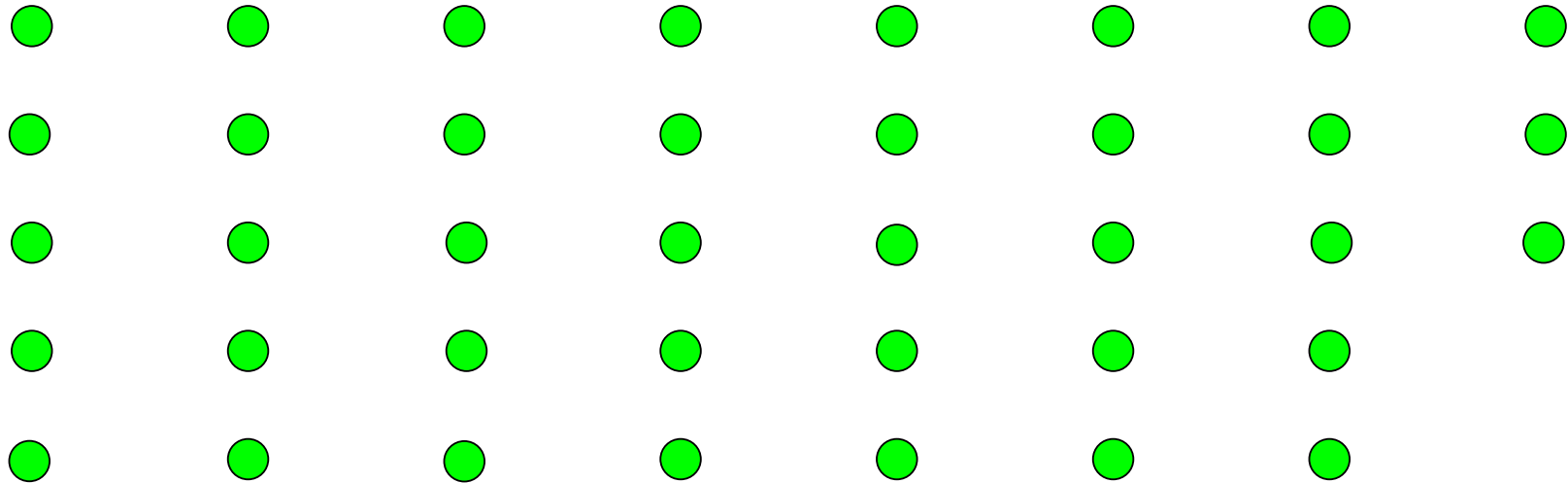
Can we do it in linear time?

Yes, selection is more easy than sorting

Deterministic linear selection

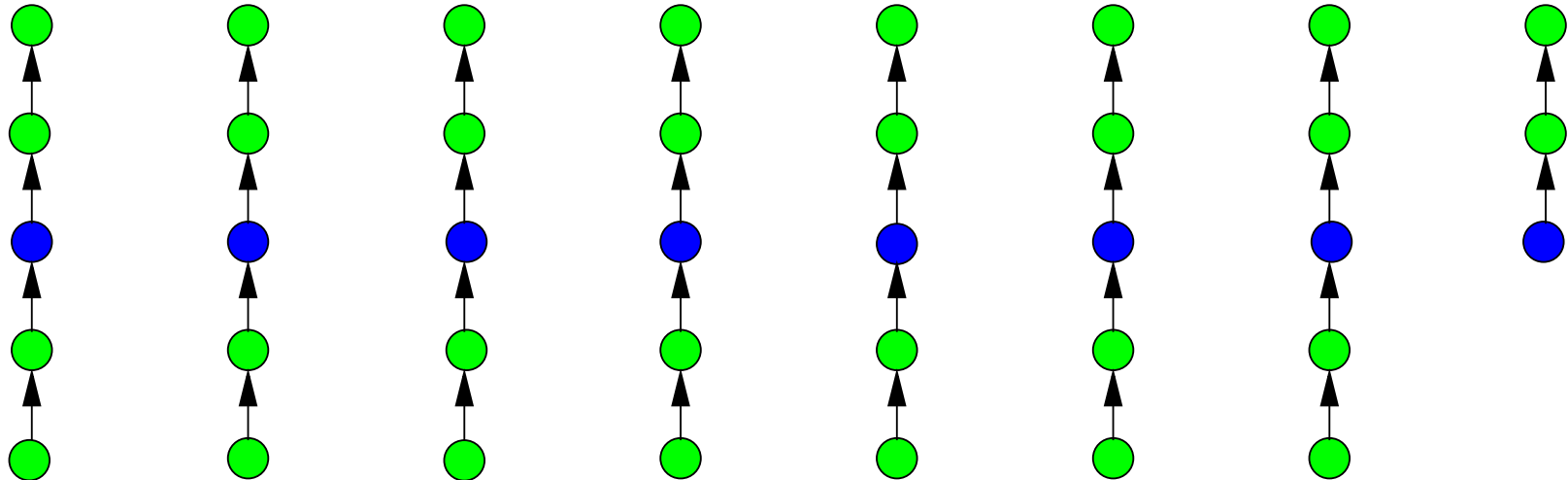
Generate deterministically a good split element x .

Divide the n elements in $\lfloor n/5 \rfloor$ groups, each with 5 elements (+ possible one group with < 5 elements).



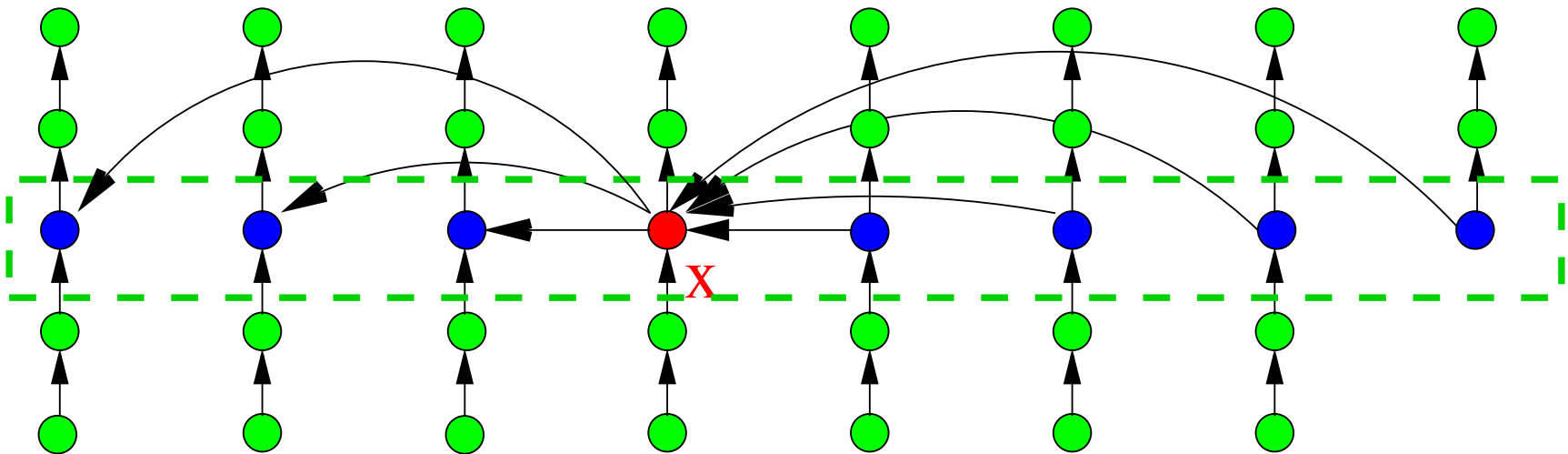
Deterministic linear selection.

Sort each set to find its median, say x_i . (Each sorting needs 5 comparisons, i.e. $\Theta(1)$) Total: $\lceil n/5 \rceil$



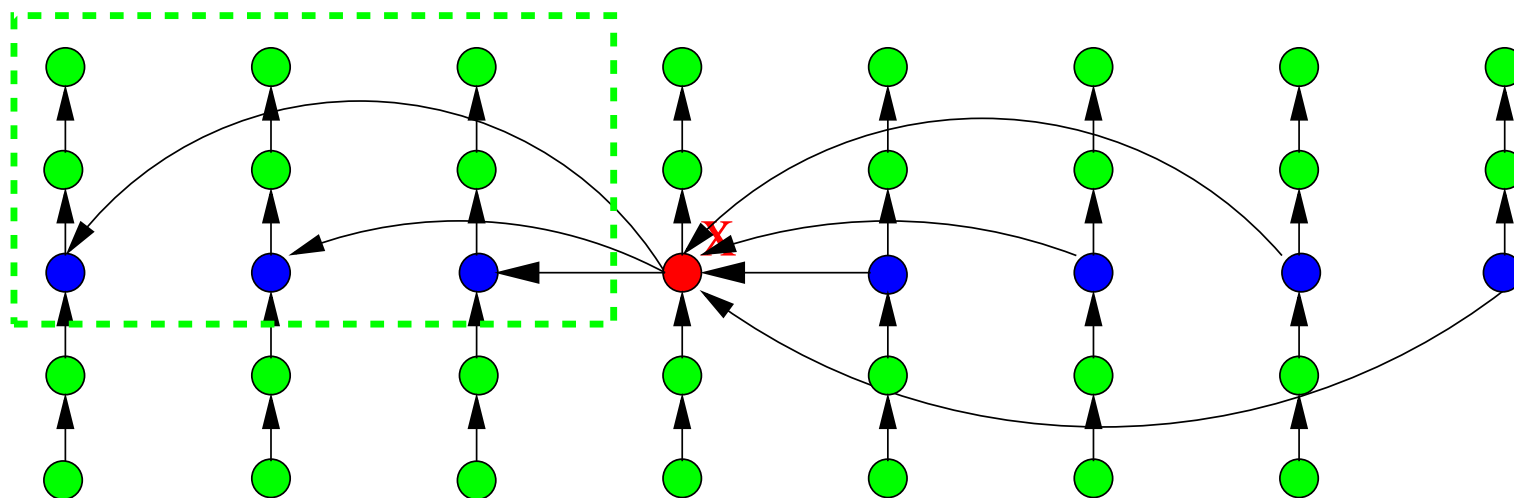
Deterministic linear selection.

- Use recursively **Select** to find the median x of the medians $\{x_i\}, 1 \leq i \leq \lceil n/5 \rceil$.
- Using **Partition** function taking as pivot the median of the medians x_i , partition the input array around x_i . Let x_i be the k -th element of the array after partitioning, so that there are $k - 1$ elements on the low side of the partition and $n - k$ elements on the high side.



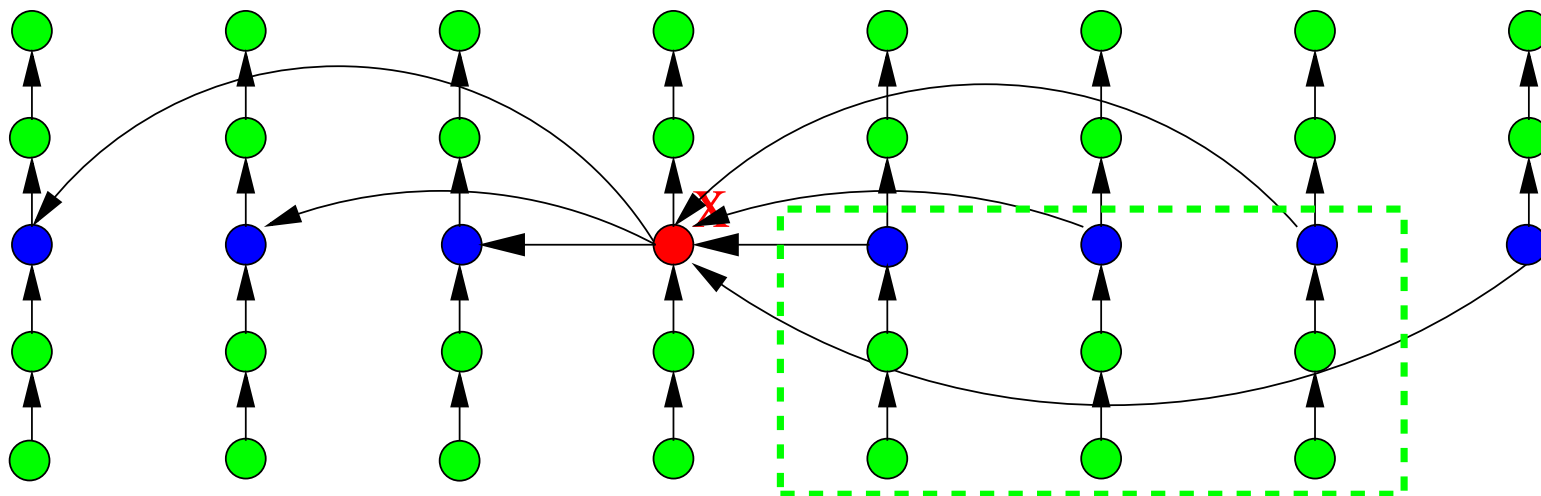
Deterministic linear selection.

At least $3(\frac{1}{2}\lfloor n/5 \rfloor) = \lfloor 3n/10 \rfloor$ of the elements are $\leq x$.



Deterministic linear selection.

At least $3(\frac{1}{2}\lfloor n/5 \rfloor) = \lfloor 3n/10 \rfloor$ of the elements are $\geq x$.



The deterministic algorithm

Select (A, i)

- 1.- Divide the n elements into $\lfloor n/5 \rfloor$ groups of 5 $O(n)$ plus a possible extra group with < 5 elements
- 2.- Find the **median** by insertion sort, and take the middle element
- 3.- Use **Select** recursively to find the median x of the $\lfloor n/5 \rfloor$ medians
- 4.- Use **Partition** the elements under consideration around x .
Let $k = \text{rank of } x$
- 5.- **if** $i = k$ **then**
 return x
 else if $i < k$ **then**
 use **Select** to find the i -th smallest in the left
 else
 use **Select** to find the $i - k$ -th smallest in the right
 end if

Example: Find the median

Get the median ($\lfloor (n + 1)/2 \rfloor$) on the following input:

3 13 9 4 5 1 15 12 10 2 6 14 8 11 17

3	1	6
4	2	8
5	10	11
9	12	14
13	15	17

PARTITION around 10:

3 4 5 9 1 2 6 8 10 13 12 15 11 14 17

To get the 8th element (median)

call SELECT on 3 4 5 9 1 2 6 8 to get 3 1 2 4 5 9 6 8

and iterate until getting the median

The deterministic algorithm

Select (A, i)

- 1.- Divide the n elements into $\lfloor n/5 \rfloor$ groups of 5 $O(n)$
plus a possible extra group with < 5 elements
- 2.- Find the **median** by insertion sort, and take
the middle element $O(n)$
- 3.- Use **Select** recursively to find the median x of the $\lfloor n/5 \rfloor$
medians $T(n/5)$
- 4.- Use **Partition** around x . $O(n)$
Let $k = \text{rank of } x$
- 5.- **if** $i = k$ **then**
 return x
 else if $i < k$ **then**
 use **Select** to find the i -th smallest in the left
 else
 use **Select** to find the $i - k$ -th smallest in the right
 end if

Worst case Analysis.

- ▶ As at least $\geq \frac{3n}{10}$ of the elements are $\geq x$.
- ▶ At least $\frac{3n}{10}$ elements are $< x$.
- ▶ In the worst case, step 5 calls **Select** recursively $\leq n - \frac{3n}{10} = 7n/10$. So step 5 takes time $\leq T(7n/10)$.

Therefore, we have

$$T(n) = \begin{cases} \Theta(1) & \text{if } n \leq 50, \\ T(n/5) + T(7n/10) + \Theta(n) & \text{if } n > 50. \end{cases}$$

Solving we get $T(n) = \Theta(n)$

Remarks on the cardinality of the groups

Notice: If we make groups of 7, the number of elements $\geq x$ is $\frac{2n}{7}$, which yield $T(n) \leq T(n/7) + T(5n/7) + O(n)$ with solution $T(n) = O(n)$.

However, if we make groups of 3, then

$T(n) \leq T(n/3) + T(2n/3) + O(n)$, which has a solution $T(n) = O(n \ln n)$.

Arbitrary i -order statistics

Given $A[1, \dots, n]$ we can use the median algorithm as a **black-box algorithm** to solve the i th. order statistics of A , i.e. finding the i -smaller element in A .

1. Find the median m .
2. Partition the array based on that median:
 - 2.1 If i is less than half the length of the original array, recurse on the first half,
 - 2.2 if i is exactly half the length of the array, return the founded median.